

PROYECTO FINAL - PRIMERA ENTREGA

KineSys

Sistema de Gestión de Clínica de Fisioterapia

Documento actualizado de descripción y pruebas del software

Estudiantes:

Eckard Antony Rodriguez Gutierrez

Luis Alejandro Narvaez Teran

Docente: Maribel Romero Mestre

Universidad Popular del Cesar - Sede Sabanas

Ingeniería de Sistemas

Valledupar - Cesar

2026

Versión adelantada el 26 de abril de 2026 con base en la plantilla "Proyecto final - Pruebas", el PDF inicial y las pruebas implementadas en el repositorio.

Tabla de contenido

1. Primera parte: descripción del sistema	2.2 Planificación de las pruebas
1.1 Identificación del problema	2.3 Pruebas unitarias
1.2 Descripción detallada del sistema o aplicación	2.4 Pruebas de integración
1.3 Modelo de requerimientos	2.5 Pruebas de sistemas
1.4 Modelo de casos de uso	2.6 Pruebas de aceptación
1.5 Modelo de diseño del sistema	3. Conclusiones
1.6 Producto del software	4. Referencias bibliográficas
2. Segunda parte: pruebas del software	5. Pendientes y observaciones
2.1 Introducción	

1. Primera Parte: Descripción Del Sistema

1.1 Identificación Del Problema

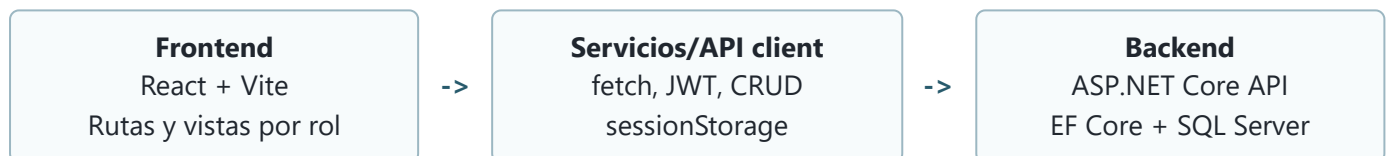
Las clínicas de fisioterapia manejan información sensible y operativa sobre pacientes, terapeutas, tratamientos, citas, disponibilidad de salas, equipos, autorizaciones, facturación y evolución clínica. Cuando estos procesos se llevan en hojas sueltas, agendas manuales o archivos aislados, aumentan los riesgos de duplicidad, errores de programación, pérdida de trazabilidad y demoras para consultar el estado real de la atención.

KineSys responde a esta necesidad mediante una plataforma web que centraliza la gestión clínica y administrativa, reduce el trabajo manual, permite consultar información por rol y fortalece el control sobre citas, pacientes, terapeutas y recursos de la clínica.

1.2 Descripción Detallada Del Sistema O Aplicación

KineSys es una aplicación web para la gestión de una clínica de fisioterapia. El frontend está construido con React y Vite; el backend usa ASP.NET Core Web API con Entity Framework Core y SQL Server; la autenticación usa JWT y el sistema incluye soporte para notificaciones en tiempo real con SignalR.

La aplicación automatiza procesos de registro y consulta de pacientes, administración de terapeutas y especialidades, gestión de citas, disponibilidad, tratamientos, salas, equipos, reportes, auditoría, facturación y vistas diferenciadas para administrador, terapeuta y paciente. El desarrollo observado sigue una estrategia incremental: se agregan módulos por funcionalidad y se respaldan comportamientos críticos con pruebas unitarias e integración.



1.3 Modelo De Requerimientos

Requisitos funcionales

ID	Módulo	Descripción
RF01	Autenticación	Permitir inicio y cierre de sesión con usuario, contraseña, JWT y roles del sistema.
RF02	Control de acceso	Redirigir o permitir acceso según roles Administrador, Terapeuta y Paciente.
RF03	Pacientes	Registrar, consultar, actualizar y eliminar información clínica y administrativa de pacientes.
RF04	Terapeutas	Gestionar terapeutas, licencias, especialidades, estado y datos profesionales.
RF05	Citas y agenda	Crear, listar, filtrar, confirmar, cancelar y validar disponibilidad de citas.
RF06	Tratamientos	Administrar tratamientos, protocolos, tipo de servicio, equipos requeridos y duración.
RF07	Recursos	Gestionar salas, equipos, disponibilidad y relaciones con sesiones terapéuticas.
RF08	Reportes	Generar reportes operativos sobre pacientes, citas, terapeutas y actividad de la clínica.
RF09	Notificaciones	Soportar alertas de agenda y notificaciones en tiempo real mediante SignalR.
RF10	Facturación	Registrar información asociada a facturas, copagos, EPS, seguros y autorizaciones.
RF11	Auditoría	Consultar eventos de acceso y acciones relevantes para trazabilidad administrativa.
RF12	Paneles por rol	Presentar vistas separadas para administrador, terapeuta y paciente.

Requisitos no funcionales

ID	Atributo	Descripción
RNF01	Seguridad	Usar JWT, cabecera Authorization Bearer, validación de expiración y cierre de sesión ante 401.
RNF02	Usabilidad	Interfaz web organizada por módulos y rutas, con formularios y listas orientados a operación clínica.
RNF03	Rendimiento	Construcción de producción con Vite; se detecta advertencia de chunk JS grande para optimización futura.
RNF04	Mantenibilidad	Separación por api, services, context, hooks, pages y pruebas automatizadas.
RNF05	Portabilidad	Frontend React/Vite ejecutable en Node.js; API .NET 8 y base SQL Server preparada en docker-compose.
RNF06	Integridad	EF Core define claves, relaciones, restricciones de borrado e índices únicos en datos críticos.
RNF07	Compatibilidad	Aplicación orientada a navegadores modernos y ambiente local Windows 11 usado en pruebas.
RNF08	Trazabilidad	Pruebas con salidas registradas en artifacts/pruebas y cobertura HTML/LCOV generada por Vitest.

1.4 Modelo De Casos De Uso

Actores principales: Administrador, Terapeuta y Paciente. El Administrador concentra la gestión operativa; el Terapeuta consulta agenda, pacientes, historias y tratamientos; el Paciente consulta su información y citas asociadas.

Actor	Casos de uso principales
Administrador	Iniciar sesión; gestionar pacientes; gestionar terapeutas; gestionar citas; gestionar tratamientos; gestionar equipos; gestionar salas; consultar reportes; revisar auditoría.
Terapeuta	Iniciar sesión; consultar dashboard; revisar citas próximas; consultar pacientes; registrar información clínica; revisar tratamientos y facturación asociada.
Paciente	Iniciar sesión; consultar panel del paciente; revisar citas próximas; consultar datos básicos y evolución disponible.
Sistema	Validar JWT; aplicar roles; enviar cabeceras Authorization; notificar eventos; mantener consistencia de relaciones e índices.

1.5 Modelo De Diseño Del Sistema

Componentes principales

Componente	Responsabilidad	Evidencia en repositorio
AppRouter	Define rutas públicas y de módulos administrativos, terapeuta y paciente.	kinesys/src/router/AppRouter.jsx
AuthContext	Maneja sesión, token, usuario, login, logout y recuperación desde sessionStorage.	kinesys/src/context/AuthContext.jsx
apiCliente	Centraliza solicitudes HTTP, cabecera Authorization y manejo de errores 401.	kinesys/src/api/apiCliente.js
Services	Encapsulan operaciones de negocio para citas, pacientes, terapeutas y demás entidades.	kinesys/src/services
API .NET	Expone controladores REST, autenticación JWT, SignalR y persistencia con EF Core.	api/ApiPrueba
ClinicaFisioterapiaBD	Contexto EF Core con DbSet, relaciones, índices y restricciones.	api/ApiPrueba/data/ClinicaFisioterapiaBD.cs

Relaciones principales del modelo de datos

Entidad	Relaciones o notas de diseño
Persona	Clase base con discriminador para Paciente, Terapeuta, Administrador y otros roles.
Paciente	Relacionado con EPS, seguro médico, citas, tratamientos, historial médico, notas, facturas y autorizaciones.
Terapeuta	Relacionado con citas, tratamientos, disponibilidad, especialidades, alertas y planes de tratamiento.
Cita	Relaciona paciente, terapeuta, tratamiento, sala, tipo de servicio, autorizaciones y estados de agenda.
Tratamiento	Relacionado con terapeuta, planes, protocolos, tipos de terapia y equipos requeridos.
DisponibilidadTerapeuta	Índice por terapeuta y día de semana para consultas frecuentes de agenda.

Avance de diagramas

La plantilla solicita diagramas UML y entidad-relación. En este PDF se deja el modelo textual adelantado con componentes y relaciones identificadas desde el código. Queda pendiente insertar los diagramas gráficos formales si el docente los exige como imagen.

1.6 Producto Del Software

Elemento	Detalle
Aplicación	KineSys - Sistema de Gestión de Clínica de Fisioterapia
Frontend local	cd kinesys; npm run dev
Build de producción	cd kinesys; npm run build
API local	cd api/ApiPrueba; dotnet run
Base de datos	SQL Server configurado en docker-compose.yml
Link público	Pendiente de despliegue o entrega por el equipo

2. Segunda Parte: Pruebas Del Software

2.1 Introducción

Esta sección presenta el plan, diseño, ejecución y evaluación de pruebas del sistema KineSys. Se actualiza el documento inicial incorporando la implementación real de Jest, Vitest y pruebas xUnit existentes en el backend.

El enfoque de pruebas combina caja negra para validar entradas, salidas y flujos de usuario, y caja blanca para verificar caminos internos, manejo de errores, servicios, hooks, contexto de autenticación, cliente API y generación de JWT.

9/9

Archivos Vitest aprobados

98/98

Pruebas frontend aprobadas

71.54%

Cobertura de sentencias

38/39

Pruebas backend aprobadas

2.2 Planificación De Las Pruebas

Objetivos

Objetivo general: verificar y validar que KineSys cumple los comportamientos críticos de autenticación, control de acceso, gestión de pacientes, terapeutas, citas y comunicación con API, reduciendo defectos en los flujos principales antes de la entrega.

- Verificar que el login maneja credenciales válidas, campos vacíos, respuestas 401, errores 500 y roles no reconocidos.
- Validar que los roles del sistema se usan correctamente para permitir o bloquear rutas protegidas.
- Comprobar que los servicios de pacientes, terapeutas y citas transforman datos y llaman los endpoints esperados.
- Verificar que apiCliente agrega Authorization cuando corresponde y limpia la sesión ante expiración.
- Medir la cobertura automatizada y dejar evidencia de ejecución en consola.
- Identificar pendientes reales para pruebas externas de rendimiento, seguridad y aceptación.

Alcance

Módulo a probar	Objetivo de la prueba
Autenticación y roles	Login, token JWT, persistencia de sesión, expiración y rutas protegidas
Pacientes	Servicios de listado, creación, actualización y eliminación; filtrado por rol Paciente
Terapeutas	Servicios de listado, creación, actualización y eliminación; filtrado por rol Terapeuta
Citas	CRUD, filtros por paciente/terapeuta/estado, resumen y verificación de solapamiento
API cliente	Métodos getAll, getByld, create, update, delete y customRequest con cabeceras
Constantes y JWT	Roles compartidos, validación de rol y lectura de claims ASP.NET
Frontend build	Compilación de producción con Vite
Backend API .NET	Pruebas xUnit existentes sobre CitasController, modelo Cita y JwtTokenGenerator

Estrategias De Pruebas

Nivel	Técnicas	Modelos / estrategias / tipos	Herramientas
Unitarias	Caja negra y caja blanca	Clases de equivalencia, valores límite, mocks de fetch/sessionStorage, render de componentes	Vitest, Jest, Testing Library
Integración	Incremental y basada en hilos	Integración Contexto -> servicios -> apiCliente; controlador .NET -> EF InMemory	Vitest/Jest, xUnit, EF Core InMemory
Sistema	Caja negra	Build de producción, seguridad de token, portabilidad local, revisión de cobertura	Vite, V8 coverage, consola
Aceptación	Caja negra	Escenarios por rol y flujo principal de negocio; avance documental con criterios esperados	Manual/pendiente Selenium o Katalon

Ambiente De Pruebas

Equipo	Especificaciones técnicas
Equipo de pruebas	Windows 11 Home Single Language 64 bits, Intel Core i5-1235U, 10 núcleos, 12 procesadores lógicos, 15.7 GB RAM
Repositorio local	C:\Users\carde\Documents\ProgramacionMovil\KineSys

Software	Versión / estado
Node.js	v24.14.0
npm	11.9.0
React	19.1.1
Vite	7.x en proyecto; build ejecutado con Vite 7.2.2
Vitest	4.1.5
Jest	30.3.0
Testing Library React	16.3.2
.NET SDK	10.0.103; API objetivo net8.0
SQL Server	Configurado por docker-compose con imagen mcr.microsoft.com/mssql/server:2022-latest
Docker	No disponible en esta máquina al momento de ejecutar pruebas

2.3 Pruebas Unitarias

Análisis De Las Pruebas

Las pruebas unitarias automatizadas se concentran en componentes funcionales de bajo acoplamiento: utilidades JWT, constantes de roles, cliente API, servicios de entidades, AuthContext, hook de autorización y formulario de login.

Archivo de prueba	Casos	Componente validado	Resultado
src/api/apiCliente.test.js	10	Cliente HTTP, token global, sessionStorage, CRUD y manejo 401	Aprobado
src/context/AuthContext.test.jsx	7	Estado inicial, recuperación de sesión, login, logout y uso fuera del proveedor	Aprobado
src/hooks/useRequireAuth.test.jsx	6	Redirecciones por autenticación y roles permitidos	Aprobado
src/services/citasService.test.js	18	Listado, creación, eliminación, filtros, resumen y disponibilidad de citas	Aprobado
src/services/pacientesService.test.js	12	CRUD de pacientes, rol 1, registro y errores del API	Aprobado
src/services/terapeutasService.test.js	12	CRUD de terapeutas, rol 2, activos y endpoint de registro	Aprobado
src/test/unitarias/login.equivalencia.test.jsx	7	Clases de equivalencia del login por rol, campos vacíos y errores	Aprobado
src/utils/constants.test.js	14	Roles del frontend, hasRole y hasAnyRole	Aprobado
src/utils/jwt.test.js	12	Decodificación JWT, claims ASP.NET y expiración	Aprobado

Análisis Por Clases De Equivalencia - Login

Campo / condición	Clases válidas	Clases inválidas
Usuario	Texto no vacío con usuario existente	Vacío; usuario inexistente; formato no registrado
Contraseña	Texto no vacío asociado al usuario	Vacía; incorrecta; error de servidor
Rol recibido	Administrador, Terapeuta, Paciente	Rol no reconocido o sin claims suficientes
Respuesta API	200 con token JWT y datos de usuario	401 credenciales inválidas; 500 error servidor

Casos Por Valores Límite

Campo	Dato de entrada	Escenario	Resultado esperado
Usuario	cadena vacía	Límite inferior inválido	Error/submit bloqueado
Usuario	cadena no vacía	Valor válido	Se permite intento de login
Contraseña	cadena vacía	Límite inferior inválido	Error/submit bloqueado
Contraseña	cadena no vacía	Valor válido	Se permite intento de login
Token JWT	sin payload o formato inválido	Formato inválido	decodificarToken devuelve null
Token JWT	exp menor al tiempo actual	Token expirado	tokenExpirado devuelve true
Token JWT	exp futuro	Token vigente	tokenExpirado devuelve false

Pruebas Del Camino Básico - Login

Método o flujo validado: envío del formulario Login.jsx, invocación de login() desde AuthContext, recepción de token, extracción de datos con jwt.js y redirección según rol.

Camino	Datos de entrada	Escenario	Resultado
C1	Usuario y contraseña válidos; rol Administrador	Respuesta 200 con token	Sesión guardada y acceso al módulo administrativo
C2	Usuario y contraseña válidos; rol Terapeuta	Respuesta 200 con token	Sesión guardada y acceso a /dashboard
C3	Campo usuario vacío	Validación de formulario	No se ejecuta login()
C4	Contraseña vacía	Validación de formulario	No se ejecuta login()
C5	Credenciales inválidas	Respuesta 401	Mensaje de error y usuario no autenticado
C6	Error interno	Respuesta 500 o fetch rechazado	Mensaje de error controlado
C7	Rol no reconocido	Token/respuesta con rol fuera de catálogo	No hay caída de interfaz

Diseño De Casos De Prueba

ID	Caso	Datos de prueba	Resultado esperado	Resultado obtenido
CE-P01	Administrador válido	usuario admin + contraseña correcta	login() se invoca con valores exactos y permite sesión	Aprobado
CE-P02	Terapeuta válido	usuario terapeuta + contraseña correcta	login() autentica y redirige al dashboard del terapeuta	Aprobado
CI-P03	Usuario vacío	usuario vacío + contraseña	validación HTML5 evita llamada a login()	Aprobado
CI-P04	Contraseña vacía	usuario + contraseña vacía	validación HTML5 evita llamada a login()	Aprobado
CI-P05	Credenciales inválidas	respuesta 401	se muestra mensaje de error	Aprobado
CI-P06	Error servidor	respuesta 500	se muestra error genérico controlado	Aprobado
CI-P07	Rol no reconocido	rol fuera de catálogo	el flujo termina sin caída de la interfaz	Aprobado

Ejecución Y Evaluación De Pruebas Unitarias

Herramienta	Comando	Resultado
Jest	npm run test:jest	Test Suites: 9 passed, 9 total. Tests: 98 passed, 98 total. Time: 5.202 s.
Vitest	npm run test	Test Files: 9 passed. Tests: 98 passed. Duration: 6.88 s.
Vitest Coverage	npm run test:coverage	98 pruebas aprobadas con cobertura V8.

Archivo / grupo	Sentencias	Ramas	Funciones	Líneas
Total frontend	71.54%	71.24%	74.69%	71.42%
api/apiCliente.js	93.33%	82.35%	80.00%	93.33%
context/AuthContext.jsx	97.95%	70.00%	83.33%	100.00%
hooks/useRequireAuth.js	91.30%	61.53%	100.00%	100.00%
pages/Login.jsx	89.65%	77.27%	66.66%	89.65%
services/pacientesService.js	79.31%	33.33%	100.00%	78.57%
services/citasService.js	48.99%	60.71%	61.11%	49.29%

Resumen de ejecución frontend:

Jest: 9 suites aprobadas, 98 pruebas aprobadas.

Vitest: 9 archivos aprobados, 98 pruebas aprobadas.

Coverage V8: Statements 71.54%, Branches 71.24%, Functions 74.69%, Lines 71.42%.

2.4 Pruebas De Integración

Estrategia De Pruebas Incrementales

Se aplica integración incremental desde utilidades y servicios hacia componentes de mayor nivel. Primero se verifican utilidades de token y roles, luego apiCliente, servicios de dominio, AuthContext, hooks de protección y finalmente flujos de login y citas.

ID	Componentes integrados	Datos / condición	Resultado esperado	Resultado obtenido
INT-01	AuthContext + API Personas/Login + jwt.js	Credenciales válidas	Token guardado en sessionStorage y usuario autenticado	Aprobado en AuthContext.test.jsx
INT-02	useRequireAuth + React Router + AuthContext	Usuario sin sesión	Redirección a /login	Aprobado en useRequireAuth.test.jsx
INT-03	apiCliente + sessionStorage	Token existente sin token global	Cabecera Authorization Bearer incluida	Aprobado en apiCliente.test.js
INT-04	citasService + disponibilidad	Crear cita con idDisponibilidad	Crea cita y actualiza disponibilidad; revierte si falla	Aprobado en citasService.test.js
INT-05	pacientesService + Register	Datos válidos de paciente	Envía user/password igual al documento y rol Paciente	Aprobado en pacientesService.test.js
INT-06	terapeutasService + Register/Update	Datos válidos de terapeuta	Asigna/preserva rol Terapeuta y estado activo	Aprobado en terapeutasService.test.js

Estrategia De Pruebas Basadas En Hilos

Se organizan hilos funcionales que atraviesan varias capas del sistema: autenticación, agenda/citas, administración de personas y seguridad del cliente API.

Hilo	Secuencia	Resultado esperado	Estado
Hilo autenticación	Login -> token JWT -> AuthContext -> rutas protegidas	Usuario autenticado con rol correcto accede; rol incorrecto redirige	Aprobado
Hilo agenda/citas	Listar citas -> filtrar -> verificar disponibilidad -> crear/eliminar	Detecta solapamientos, ignora canceladas y libera disponibilidad	Aprobado
Hilo administración de personas	Listar personas -> filtrar por rol -> crear paciente/terapeuta -> actualizar/eliminar	Servicios separan Paciente rol 1 y Terapeuta rol 2	Aprobado
Hilo seguridad del API cliente	Token global/sessionStorage -> solicitud fetch -> respuesta 401	Elimina sesión y dispara evento auth-token-expired	Aprobado

Integración Backend .NET

El backend contiene un proyecto ApiPrueba.Tests con xUnit, FluentAssertions, Moq y EF Core InMemory. Se ejecutó dotnet test sobre la solución completa.

Grupo	Cantidad	Propósito	Estado
CitasControllerTests	14 escenarios declarados	GET/POST/PUT/DELETE con EF Core InMemory	Ejecutado
JwtTokenGeneratorTests	11 escenarios declarados	Formato JWT, claims, rol, expiración y unicidad	Ejecutado
CitaModelTests	9 escenarios declarados	Atributos, valores por defecto, enum Rol y nulabilidad	Ejecutado con 1 fallo

Resultado backend

dotnet test ejecutó 39 pruebas: 38 superadas y 1 fallida. La falla no corresponde a una caída funcional de API sino a una expectativa desactualizada en CitaModelTests: el test espera exactamente 3 roles, pero el enum Rol actual contiene 6 roles.

2.5 Pruebas De Sistemas

Las pruebas de sistema se adelantan con evidencia automatizada disponible en el entorno local. Las pruebas externas de JMeter, OWASP ZAP, Selenium o Katalon quedan planteadas para una siguiente iteración porque esas herramientas no se encuentran configuradas en el repositorio ni instaladas en esta máquina.

Tipo de prueba	Nombre / foco	Herramienta	Resultado
Seguridad	Token JWT, roles, Authorization Bearer, limpieza de sesión ante 401	Vitest/Jest/xUnit	Automatizado aprobado en frontend; xUnit con observación en enum Rol
Rendimiento	Build de producción y tamaño de paquetes	Vite build	Build exitoso en 14.19 s; advertencia por chunk JS de 1,858.38 kB
Usabilidad	Login, rutas por rol y formularios principales	Testing Library + revisión documental	Flujos principales del login aprobados; pruebas con usuarios reales pendientes
Portabilidad	Ejecución en Windows 11, Node, npm y .NET	Build y pruebas locales	Frontend compila y pruebas pasan; Docker no disponible localmente

Build de producción:
npm run build
Vite build exitoso en 14.19 s.
2878 módulos transformados.
CSS: 287.18 kB (gzip 42.07 kB).
JS: 1,858.38 kB (gzip 526.32 kB).
Observación: Vite advierte que algunos chunks superan 500 kB; se recomienda code-splitting en una iteración futura.

2.6 Pruebas De Aceptación

Se diseñan casos de aceptación centrados en los requisitos más visibles para el usuario final. Parte de estos casos ya está respaldada por pruebas automatizadas; los escenarios que dependen de interacción completa con navegador y datos reales quedan pendientes para ejecución manual o automatización E2E.

ID Caso	Requisito asociado	Escenario de negocio	Pasos de ejecución	Resultado esperado	Resultado obtenido
CA-01	RF01/RF02	Ingreso de usuario terapeuta	Abrir /login, ingresar credenciales válidas, enviar formulario	Acceso al panel de terapeuta	Aprobado por pruebas automatizadas del flujo
CA-02	RF03	Registrar paciente desde administración	Ingresar datos válidos y enviar	Paciente creado con rol 1 y credenciales por documento	Diseñado; ejecución manual pendiente
CA-03	RF04	Registrar terapeuta desde administración	Ingresar datos profesionales válidos y guardar	Terapeuta creado con rol 2 y estado activo	Diseñado; ejecución manual pendiente
CA-04	RF05	Crear cita sin conflicto	Seleccionar paciente, terapeuta y horario disponible	Cita creada y disponibilidad actualizada	Cubierto por servicio; ejecución UI pendiente
CA-05	RNF01	Sesión expirada	Forzar respuesta 401 en solicitud autenticada	Sesión limpiada y evento de expiración emitido	Aprobado en apiCliente.test.js

3. Conclusiones

1. Las pruebas unitarias frontend quedaron consolidadas con Jest y Vitest: 98 casos pasan correctamente en 9 archivos de prueba.
2. La cobertura automatizada del frontend alcanza 71.54% en sentencias y 71.42% en líneas, suficiente como avance para una primera entrega, con oportunidad de mejorar en services/citasService.
3. El flujo de autenticación es el componente mejor respaldado: se validan credenciales, sesión persistida, claims JWT, expiración, roles y rutas protegidas.
4. Las pruebas de integración muestran que apiCliente, servicios, AuthContext y hooks se comunican correctamente bajo escenarios exitosos y de error.
5. El backend posee una base de pruebas xUnit útil para controlador de citas, modelo Cita y generación JWT; sin embargo, debe ajustarse una prueba desactualizada sobre el enum Rol.
6. La aplicación compila para producción con Vite; el principal hallazgo técnico de sistema es el tamaño del bundle JavaScript, que conviene optimizar con carga dinámica o manualChunks.
7. Quedan pendientes las evidencias gráficas solicitadas por la plantilla: diagramas UML finales, pantallazos de interfaz, pruebas con JMeter, OWASP ZAP y pruebas E2E con Selenium/Katalon.

4. Referencias Bibliográficas

- Repositorio local KineSys: código fuente del frontend React/Vite y backend ASP.NET Core.
- Plantilla proyecto final - Pruebas.doc, Universidad Popular del Cesar.
- Documentación oficial de React, Vite, Vitest, Jest, Testing Library, ASP.NET Core, Entity Framework Core y xUnit.
- Salidas de ejecución generadas en artifacts/pruebas: jest-output.txt, vitest-output.txt, coverage-output.txt, build-output.txt y dotnet-test-output.txt.

5. Pendientes Y Observaciones

Elemento	Motivo	Avance dejado en este documento
Diagramas UML formales	No hay imágenes editables en el repositorio	Se deja diseño textual y relaciones principales adelantadas
Pantallazos de interfaz	No se levantó servidor visual para capturas	Se documentan rutas, módulos y evidencias de consola
JMeter	Herramienta no instalada	Prueba de rendimiento queda planteada; build Vite ejecutado como avance
OWASP ZAP	Herramienta no instalada	Prueba de seguridad queda planteada; autenticación y token cubiertos por tests
Selenium/Katalon	No configurado en el proyecto	Pruebas de aceptación quedan diseñadas para ejecución manual o futura automatización
Backend xUnit	1 prueba fallida por enum Rol	La prueba espera 3 roles, pero el modelo contiene 6: Paciente, Terapeuta, Administrador, Recepcionista, Contador y Supervisor

Eckard Antony Rodriguez Gutierrez

Luis Alejandro Narvaez Teran